

IP Routing Analysis Using Wireshark

Computer Networks Practical Record

1. Aim

The aim of this experiment is to study and analyze IP routing using Wireshark along with Windows networking utilities. The experiment observes how packets travel from the local system to a remote destination (8.8.8.8) through one or more intermediate routers, and correlates command-line output (ping, tracert, route print, ipconfig) with the actual packets captured in Wireshark.

2. Objective

- Understanding the basic concept of IP routing.
- Understanding the role of a default gateway.
- Examining the Windows routing table.
- Capturing ICMP packets using Wireshark.
- Understanding TTL and its role in traceroute.
- Identifying intermediate routers between the source and destination.
- Observing ICMP Echo Request and Echo Reply packets.
- Understanding ICMP Time-to-Live Exceeded messages.
- Verifying network configuration using ipconfig.
- Correlating command-line results with Wireshark packet captures.

3. Tools and Requirements

- Windows operating system
- Wireshark 4.6.8
- Npcap
- Command Prompt / PowerShell
- ping
- tracert
- route
- ipconfig
- Internet connection

Wireshark is used to capture and inspect the network packets that are generated when commands such as ping and tracert are executed. Wireshark itself cannot capture traffic directly from the network adapter on Windows; it depends on a packet-capture driver. Npcap provides this driver and interface, so it was selected during installation to enable normal IP/network traffic capture.

USBPcap, which is an optional component offered during Wireshark installation for capturing USB traffic, was not required for this experiment. Since the objective was to analyze IP routing and ICMP traffic over the network adapter and not USB communication, this component was left unselected.

4. Introduction / Theory

IP Routing

IP routing is the process by which a packet is forwarded from a source host to a destination host, possibly across multiple networks. When the destination is not on the same local network as the sender, the packet cannot be delivered directly. Instead, it is passed to a router, which examines the destination IP address and forwards the packet toward the next network on the path. This process is repeated by each router until the packet reaches its destination.

Routing Table

Every host maintains a routing table that lists known network destinations and the corresponding gateway or interface to use for each. When a host needs to send a packet, it compares the destination IP address against the entries in its routing table to decide whether the destination is on the local network or must be sent through a gateway.

Default Gateway

The default gateway is the router used to forward traffic when no more specific route exists in the routing table for the destination network. In this experiment, all traffic destined for remote networks (such as 8.8.8.8) is passed to the default gateway, since the destination does not belong to the local subnet.

Network and Subnet Mask

The subnet mask determines which portion of an IP address represents the network and which portion represents the host, and is used to decide whether a destination address lies within the local network or must be reached through a gateway. A host compares its own network address with the destination network address; if they match, the destination is local, otherwise the packet is routed through the default gateway.

TTL (Time to Live)

TTL is a field in the IP header that limits how long a packet can remain in the network. Every router that forwards a packet decrements the TTL value by one. If a router decrements the TTL to zero, it discards the packet and sends back an ICMP Time-to-Live Exceeded message to the sender.

TTL therefore prevents packets from circulating indefinitely and is also the mechanism that traceroute relies on to discover intermediate routers.

ICMP

ICMP (Internet Control Message Protocol) is used to report errors and exchange control information between hosts and routers. In this experiment ICMP is used in three ways: ping uses ICMP Echo Request and Echo Reply messages to test reachability, tracert uses ICMP probes with increasing TTL values, and routers along the path send ICMP Time-to-Live Exceeded messages when a probe's TTL reaches zero.

Traceroute

The Windows tracert utility discovers the routers on the path to a destination by sending a series of probes with progressively increasing TTL values, starting at 1. The first probe, with TTL = 1, is discarded by the first router on the path, which returns a Time-to-Live Exceeded message identifying itself. The next probe, with TTL = 2, is discarded by the second router, and so on. This continues until a probe reaches the final destination, revealing the complete path one hop at a time.

5. Wireshark Installation and Setup

Wireshark was installed on the Windows machine used for this experiment. During installation, Npcap was selected as it provides the packet-capture driver required for Wireshark to capture live traffic from the network adapter. USBPcap was not selected, since the experiment involves only IP/network traffic and does not require USB packet capture.

After installation, Wireshark was opened and the list of available capture interfaces was reviewed. The Wi-Fi interface was selected as the capture interface because it was the active network interface being used by the system to connect to the internet at the time of the experiment. All subsequent captures (ping and traceroute analysis) were performed on this interface.

6. Network Configuration

Before generating traffic, the network configuration of the local host was verified using the following command:

- *ipconfig*

The ipconfig command displays the IP configuration of all network adapters on the system, including the assigned IPv4 address, subnet mask, and default gateway. It was used here to confirm the local host's identity on the network before analyzing how its packets are routed.

```

PS C:\Users\APPADI> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 1:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::3dd6:566f:3b89:1be0%2
    IPv4 Address. . . . . : 192.168.1.186
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.1.1

Ethernet adapter Bluetooth Network Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

```

Figure 1: Output of ipconfig showing the Wi-Fi adapter configuration

As shown in Figure 1, the active Wi-Fi adapter has the following configuration:

- IPv4 Address: 192.168.1.186
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.1.1

192.168.1.186 is the address of the local host used throughout this experiment. The subnet mask 255.255.255.0 corresponds to a /24 network, meaning the first 24 bits identify the network and the remaining 8 bits identify the host. 192.168.1.1 is the default gateway, that is, the router to which the host forwards traffic destined for networks outside its own subnet.

Based on these values, the local network can be represented as 192.168.1.0/24. Any destination address outside this range, such as 8.8.8.8, is treated as remote and is routed through the default gateway rather than delivered directly.

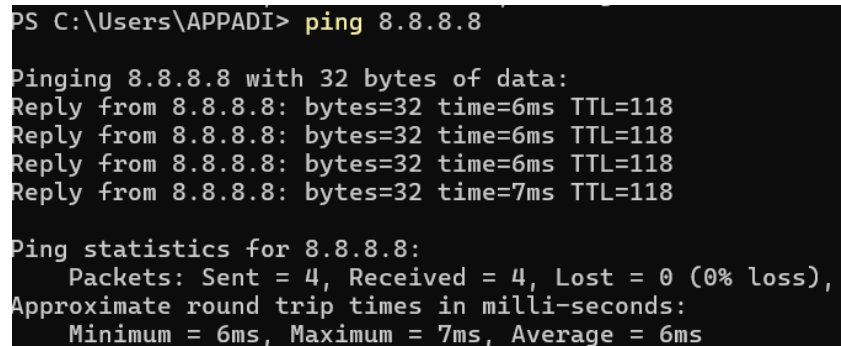
7. Testing Connectivity Using Ping

To test basic reachability to a remote host, the ping command was used with Google's public DNS server as the destination:

- `ping 8.8.8.8`

The ping command sends ICMP Echo Request packets from the local host to the specified destination and waits for ICMP Echo Reply packets in return. It was used here to confirm that the

destination 8.8.8.8 was reachable before performing the traceroute analysis, and to observe the round-trip time and TTL of the replies.



```
PS C:\Users\APPADI> ping 8.8.8.8

Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=6ms TTL=118
Reply from 8.8.8.8: bytes=32 time=6ms TTL=118
Reply from 8.8.8.8: bytes=32 time=6ms TTL=118
Reply from 8.8.8.8: bytes=32 time=7ms TTL=118

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 6ms, Maximum = 7ms, Average = 6ms
```

Figure 2: Output of ping 8.8.8.8 from the local host

As shown in Figure 2, four ICMP Echo Request packets were sent from 192.168.1.186 to 8.8.8.8, and all four received replies:

- Packets sent: 4
- Packets received: 4
- Packet loss: 0%
- Minimum round-trip time: 6 ms
- Maximum round-trip time: 7 ms
- Average round-trip time: 6 ms
- TTL of replies: 118

Zero percent packet loss confirms that the destination is reachable and that no packets were dropped along the path during this test. The low, consistent round-trip times (6-7 ms) indicate a short and stable path to the destination. A reply TTL of 118 means that the packet's TTL was decremented a number of times equal to the number of routers it passed through on its way back from 8.8.8.8, since it started from a higher initial value (commonly 128 or 64 depending on the sending host's operating system defaults).

The same exchange was then examined in Wireshark using the display filter:

- *icmp*

This filter restricts the packet list to ICMP traffic only, making it easier to isolate the Echo Request and Echo Reply packets generated by the ping command.

No.	Time	Source	Destination	Protocol	Length	Info
1309	90.752154400	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (reply in 1310)
1310	90.762621100	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=1/256, ttl=118 (request in 1309)
1311	91.755603500	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (reply in 1312)
1312	91.761389900	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=2/512, ttl=118 (request in 1311)
1327	92.769529800	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 1328)
1328	92.775661300	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=3/768, ttl=118 (request in 1327)
1355	93.784826100	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 1356)
1356	93.791461800	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=4/1024, ttl=118 (request in 1355)
1550	149.817732700	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=5/1280, ttl=128 (reply in 1551)
1551	149.824307300	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=5/1280, ttl=118 (request in 1550)
1554	150.821118700	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=6/1536, ttl=128 (reply in 1555)
1555	150.827094700	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=6/1536, ttl=118 (request in 1554)
1556	151.826044600	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=7/1792, ttl=128 (reply in 1557)
1557	151.832933900	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=7/1792, ttl=118 (request in 1556)
1564	152.837800400	192.168.1.186	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=8/2048, ttl=128 (reply in 1565)
1565	152.844977400	8.8.8.8	192.168.1.186	ICMP	78	Echo (ping) reply id=0x0001, seq=8/2048, ttl=118 (request in 1564)

Figure 3: Wireshark capture of ICMP Echo Request and Echo Reply packets for the ping test

Figure 3 shows the ICMP Echo Request packets sent from 192.168.1.186 to 8.8.8.8 with TTL = 128, and the corresponding Echo Reply packets sent back from 8.8.8.8 to 192.168.1.186 with TTL = 118. The Source and Destination columns confirm the direction of each packet, and the Info column identifies each packet as an Echo (ping) request or reply along with its identifier and sequence number.

The TTL values differ between the request and the reply because each packet starts from its own sender's initial TTL. The request, originating at the local host, starts at TTL = 128. The reply, originating at 8.8.8.8, starts from its own initial TTL and is decremented once for every router it passes through on the return path; the observed value of 118 reflects the hops traversed on that return path, consistent with the TTL = 118 reported by the ping command itself.

8. Examining the Windows Routing Table

The local routing table was examined using:

- `route print`

The route print command displays the IPv4 (and IPv6) routing table maintained by the operating system. It was used here to confirm how the host decides where to forward packets destined for both local and remote networks.

```

PS C:\Users\APPADI> route print
=====
Interface List
5...28 c5 c8 2e 30 7b .....Realtek Gaming GbE Family Controller
19...dc 45 46 16 2a 1b .....Microsoft Wi-Fi Direct Virtual Adapter
11...de 45 46 16 2a 1a .....Microsoft Wi-Fi Direct Virtual Adapter #2
2...dc 45 46 16 2a 1a .....Intel(R) Wi-Fi 6E AX211 160MHz
14...dc 45 46 16 2a 1e .....Bluetooth Device (Personal Area Network)
1.....Software Loopback Interface 1
=====

IPv4 Route Table
=====
Active Routes:
Network Destination        Netmask          Gateway           Interface        Metric
0.0.0.0                    0.0.0.0          192.168.1.1       192.168.1.186    35
127.0.0.0                  255.0.0.0        On-link           127.0.0.1        331
127.0.0.1                  255.255.255.255  On-link           127.0.0.1        331
127.255.255.255            255.255.255.255  On-link           127.0.0.1        331
192.168.1.0                 255.255.255.0    On-link           192.168.1.186    291
192.168.1.186               255.255.255.255  On-link           192.168.1.186    291
192.168.1.255               255.255.255.255  On-link           192.168.1.186    291
224.0.0.0                   240.0.0.0        On-link           127.0.0.1        331
224.0.0.0                   240.0.0.0        On-link           192.168.1.186    291
255.255.255.255             255.255.255.255  On-link           127.0.0.1        331
255.255.255.255             255.255.255.255  On-link           192.168.1.186    291
=====
Persistent Routes:
None

IPv6 Route Table
=====
Active Routes:
If Metric Network Destination      Gateway
1    331 ::1/128                On-link
2    291 fe80::/64                On-link
2    291 fe80::3dd6:566f:3b89:1be0/128
                                     On-link
1    331 ff00::/8                 On-link
2    291 ff00::/8                 On-link
=====
Persistent Routes:
None

```

Figure 4: IPv4 routing table displayed using route print

The routing table in Figure 4 contains several entries, each made up of a Network Destination, Netmask, Gateway, Interface, and Metric:

- Network Destination - the network or host address the entry applies to.
- Netmask - defines which portion of the destination address is compared against the network.
- Gateway - the next-hop router used to reach that destination, or On-link if the destination is directly reachable.
- Interface - the local IP address of the adapter used to send the packet.
- Metric - a cost value used to choose between multiple matching routes; a lower metric is preferred.

Two entries are particularly relevant to this experiment. The default route is shown as:

```
0.0.0.0          0.0.0.0          192.168.1.1      192.168.1.186
```

This entry matches any destination not covered by a more specific route (0.0.0.0/0). It states that unmatched traffic should be sent to the gateway 192.168.1.1 using the local interface

192.168.1.186. This is the entry used when the host sends traffic to 8.8.8.8, since that address does not belong to the local subnet.

The local network route is shown as:

```
192.168.1.0      255.255.255.0    On-link      192.168.1.186
```

This entry covers the local subnet 192.168.1.0/24 and is marked On-link, meaning destinations in this range are delivered directly over the local network without going through a gateway.

When the host needs to send a packet to 8.8.8.8, it compares this destination against every entry in the routing table. Since 8.8.8.8 does not match the local network entry (192.168.1.0/24), the most specific matching entry is the default route, so the packet is forwarded to the gateway 192.168.1.1 for further routing toward the internet.

9. Traceroute Analysis

To trace the actual path taken by packets to 8.8.8.8, the following command was used:

- *tracert 8.8.8.8*

The tracert command sends a series of ICMP probes with increasing TTL values toward the destination and records which router responds at each TTL. It was used here to identify the sequence of routers between the local host and 8.8.8.8.

```
PS C:\Users\APPADI> tracert 8.8.8.8

Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:

  0  0 ms   0 ms   0 ms   192.168.1.1
  1  1 ms   1 ms   14 ms  192.168.1.1
  2  15 ms  14 ms  15 ms  172.16.76.110
  3  *      *      *      Request timed out.
  4  6 ms   5 ms   4 ms   as15169.bom.extreme-ix.net [103.77.108.82]
  5  6 ms   3 ms   3 ms   142.251.76.31
  6  6 ms   3 ms   3 ms   142.250.214.99
  7  7 ms   4 ms   4 ms   dns.google [8.8.8.8]

Trace complete.
PS C:\Users\APPADI>
```

Figure 5: Output of *tracert 8.8.8.8*

The observed route, as shown in Figure 5, is:

1. 192.168.1.1
2. 172.16.76.110
3. Request timed out
4. as15169.bom.extreme-ix.net [103.77.108.82]
5. 142.251.76.31
6. 142.250.214.99
7. dns.google [8.8.8.8]

Hop 1 (192.168.1.1) is the local default gateway, and is reached first because every packet leaving the local network must pass through it. Hop 2 (172.16.76.110) is the next router along the path, reached with a TTL of 2.

Hop 3 did not respond within the probe timeout and is shown as "Request timed out". This does not necessarily indicate a routing failure at that hop; it only means that router did not send back an ICMP Time-to-Live Exceeded reply, which can happen if the router is configured not to respond to such probes or if the response was filtered. This is confirmed by the fact that the later hops did respond normally, showing that packets continued to be forwarded correctly through hop 3 even though it did not identify itself.

Hops 4, 5, and 6 (103.77.108.82, 142.251.76.31, and 142.250.214.99) are subsequent routers on the path toward Google's network, with hop 4 identifying itself as as15169.bom.extreme-ix.net, an internet exchange point. Hop 7 is the final destination, 8.8.8.8 (dns.google), confirming that the trace successfully reached the target.

The round-trip times recorded at each hop remain low throughout the trace, generally in the range of a few milliseconds, and increase only slightly moving from the local gateway toward the more distant hops. This is consistent with a short, low-latency path from the local network to the destination.

10. Wireshark Traceroute Packet Analysis

The packets generated during the traceroute were also examined in Wireshark using the same display filter:

- `icmp`

No.	icmp	Source	Destination	Protocol	Length	Info
100	192.168.1.186	192.168.1.186	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=30/7680, ttl=1 (no response found!)
109	192.168.1.1	192.168.1.186	192.168.1.186	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
110	192.168.1.186	192.168.1.186	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=31/7936, ttl=1 (no response found!)
111	192.168.1.1	192.168.1.186	192.168.1.186	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
112	192.168.1.186	192.168.1.186	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=32/8192, ttl=1 (no response found!)
113	192.168.1.1	192.168.1.186	192.168.1.186	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
123	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	124	Destination unreachable (Port unreachable)
147	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	124	Destination unreachable (Port unreachable)
201	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	124	Destination unreachable (Port unreachable)
219	192.168.1.186	192.168.1.186	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=33/8448, ttl=2 (no response found!)
220	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
221	192.168.1.186	192.168.1.186	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=34/8704, ttl=2 (no response found!)
222	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
223	192.168.1.186	192.168.1.186	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=35/8960, ttl=2 (no response found!)
224	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	138	Time-to-live exceeded (Time to live exceeded in transit)
228	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	124	Destination unreachable (Port unreachable)
234	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	124	Destination unreachable (Port unreachable)
237	192.168.1.186	192.168.1.186	192.168.1.186	ICMP	124	Destination unreachable (Port unreachable)
242	192.168.1.186	192.168.1.186	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=36/9216, ttl=3 (no response found!)

Figure 6: Wireshark capture of ICMP probes and Time-to-live exceeded responses during the traceroute

Figure 6 shows the ICMP probes sent from 192.168.1.186 to 8.8.8.8 at increasing TTL values, along with the Time-to-live exceeded responses returned by the routers along the path.

TTL = 1

The capture shows Echo (ping) request packets from 192.168.1.186 to 8.8.8.8 with ttl=1, marked "no response found!" since these are not answered by the final destination. Each of these probes is

followed by a Time-to-live exceeded packet sent from 192.168.1.1 to 192.168.1.186. Because 192.168.1.1 is the router that returns this message for the TTL = 1 probes, it is identified as the first hop on the path, matching hop 1 in the traceroute output.

TTL = 2

The capture then shows probes with ttl=2, which are answered by Time-to-live exceeded packets from 172.16.76.110. Since this router is the one that responds once the probe TTL reaches 2, it is identified as the second hop on the path, matching hop 2 in the traceroute output.

Increasing TTL

This pattern illustrates the core mechanism behind traceroute: each successive probe is sent with a TTL one greater than the last (TTL 1 to identify the first router, TTL 2 for the second, TTL 3 for the third, and so on). Every router that receives a probe decrements its TTL by one; when a router decrements the TTL to zero, it discards the packet and generates an ICMP Time-to-live exceeded message identifying itself as the sender. By observing which router responds at each TTL value, the complete path to the destination can be reconstructed one hop at a time. Destination unreachable (Port unreachable) messages are also visible in the capture; these arise separately, from probes that did reach a responding host but were sent to a closed port.

11. Correlation Between Tracert and Wireshark

The traceroute command and the Wireshark capture describe the same routing path from two different perspectives. traceroute presents the path as a simple, human-readable hop list, while Wireshark exposes the underlying ICMP packets that produce that list.

The traceroute output recorded the path as:

```
192.168.1.1 -> 172.16.76.110 -> [timeout] -> 103.77.108.82 ->  
142.251.76.31 -> 142.250.214.99 -> 8.8.8.8
```

The Wireshark capture confirms the first two hops directly:

- TTL = 1 probes receive a Time-to-live exceeded response from 192.168.1.1.
- TTL = 2 probes receive a Time-to-live exceeded response from 172.16.76.110.
- Further TTL values (3 and above) are used in the same manner to discover the subsequent hops, up to the point where the final destination replies directly.

Both methods therefore demonstrate the same underlying routing path: traceroute summarizes it as a list of hops with round-trip times, while Wireshark shows the exact ICMP request and Time-to-live exceeded packets that were exchanged to determine each of those hops.

12. Overall Packet Flow

The path observed during this experiment can be represented as follows:

```

192.168.1.186
  |
  v
192.168.1.1   (Hop 1 - Default Gateway)
  |
  v
172.16.76.110 (Hop 2)
  |
  v
[ Hop 3 - did not respond ]
  |
  v
103.77.108.82 (Hop 4)
  |
  v
142.251.76.31 (Hop 5)
  |
  v
142.250.214.99 (Hop 6)
  |
  v
8.8.8.8       (Destination)

```

This diagram represents the observed traceroute path from the local host to the destination. The third hop is shown without a specific address because it did not respond to the traceroute probes; its position in the path is nonetheless confirmed by the fact that the hops before and after it responded normally.

13. Observations

Parameter	Observed Value
Local IPv4 address	192.168.1.186
Subnet mask	255.255.255.0 (/24)
Default gateway	192.168.1.1
Destination IP	8.8.8.8 (dns.google)
Ping packet loss	0% (4 sent, 4 received)
Ping latency	Min 6 ms, Max 7 ms, Avg 6 ms
Ping reply TTL	118
First routing hop	192.168.1.1

Second routing hop	172.16.76.110
Third hop	Did not respond (request timed out)
Fourth hop	103.77.108.82 (as15169.bom.extreme-ix.net)
Fifth hop	142.251.76.31
Sixth hop	142.250.214.99
Final destination (hop 7)	8.8.8.8 (dns.google)
Wireshark ICMP observations	Echo request/reply for ping; Time-to-live exceeded for traceroute probes
TTL-based route discovery	TTL = 1 identified 192.168.1.1; TTL = 2 identified 172.16.76.110

14. Result

The experiment successfully demonstrated IP routing from the local Windows host to the destination 8.8.8.8. The routing table, ping, tracert, and Wireshark packet capture were together used to analyze default gateway selection, packet forwarding, the intermediate routers on the path, TTL behaviour, the ICMP messages involved, and end-to-end connectivity between the source and destination.

15. Conclusion

This experiment showed how a host uses its routing table to decide where to send outgoing traffic. When a destination lies outside the local subnet, as with 8.8.8.8 in this experiment, the packet is forwarded first to the default gateway and then passed along by successive routers until it reaches its destination.

The ping test confirmed basic reachability and provided round-trip time and TTL information, while route print showed the exact table entries the host consults to make forwarding decisions. The tracert command then revealed the individual routers along the path by sending probes with progressively increasing TTL values, and Wireshark provided packet-level visibility into this process, showing the actual ICMP Echo Request, Echo Reply, and Time-to-live exceeded packets involved. Together, these tools demonstrated, from two complementary perspectives, how TTL and ICMP messages allow a path of otherwise invisible intermediate routers to be identified and confirmed.